Activity 3 Report: Cross-Group Deployment and Verification

COMP 018 Database Administration | Project tested: BrewPOS (Cafe Point-of-Sale) (Spring Boot + PostgreSQL) | Date of testing: 6 November 2026

1. Cover and links

Item	Value
Testing group	G08: Borreta, Joshua; Bola, Martin Lawrence; Juatchon, Joaquin Phillip; Marcos, Karl Raphael; Pacheco Jr., Sancho; Vera Cruz, Euan Joseph
Project tested	BrewPOS (Cafe Point-of-Sale)
Owning group	Group 4: Olivarez, Charles Kean E.; Infantado, Gerwin G.; Pocsidio, Jhon Louie F.; Abangan, SJ Kaizen M.; Bartolata, Redge Raindel N.
Repository	https://github.com/Nycri/brewpos
Tracker	https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System/tree/main/other_group_tests
Evidence files	a3_deploy.txt, a3_app.txt, a3_projectverify.txt, a3_mvn_test_full.txt, screenshots

This is a full re-test. The database was dropped and rebuilt from the repository files, and every result below comes from this run.

2. Test environment

Item	Value
Operating system	Windows 11
PostgreSQL	18.6 (x86_64 Windows), psql on PATH
Java	Temurin OpenJDK 21.0.13 (the README asks for 17 or newer)
Build tool	Maven Wrapper included in the repository (mvnw.cmd)
Spring Boot	4.1.1, from pom.xml
Database	grp_brewposdb, created by the tester
Browser	Web browser on Windows (screenshots in Figures 1 to 6)

3. Installation procedure and result

The README was followed as written (Option B, PowerShell and psql). Nothing outside the repository was used.

Step	Action	Result
1	Extracted the zip, checked pom.xml, mvnw.cmd and .env.example	All present at the project root
2	CREATE DATABASE grp_brewposdb	OK
3	psql -f database/schema.sql	OK. 5 tables and 2 indexes, no errors
4	psql -f database/seed.sql	OK. 2 staff, 3 categories, 11 menu items, 5 orders with 9 lines

5	Ran seed.sql a second time	Row counts unchanged, so the README claim that it is safe to repeat holds
6	Ran the project's own database/03_verification_queries.sql	126 lines of output, no errors
7	Copied .env.example to .env and entered the postgres password	OK
8	.\mvnw.cmd test	19 tests, 0 failures, 0 errors, BUILD SUCCESS in 17 seconds
9	.\mvnw.cmd spring-boot:run	Started BrewPosApplication in 3.9 seconds, listening on port 8081

The test run prints an ERROR line from GlobalExceptionHandler. It comes from a test that forces a server error on purpose to check that the response does not leak details, so it is not a fault. The tests leave no rows behind (counts before and after are 2, 3, 11, 5 and 9), but they do use up sequence numbers: the first real order after the tests was #10.

4. Database verification

Check	Result
Tables	Exactly 5: staff, menu_category, menu_item, order_ticket, order_line_item
Constraints	5 check, 4 foreign key, 5 primary key (order_line_item uses order_id + item_id), 2 unique, 18 not null
Business rules in the database	price >= 0, quantity > 0, sold_price >= 0, payment method CASH or ONLINE, staff role CASHIER or MANAGER, unique category name, unique item name per category
Indexes	9, including idx_order_ticket_timestamp and idx_order_line_item_item_id
Sequences	4
Triggers and views	None. The project does all its rules with constraints and application code
Account used by the running app	postgres, a superuser (the README example puts it in .env). The 10 pooled connections are all postgres
Object owner	postgres for all 5 tables; database ACL is empty (default)

5. Application checks

The TESTING_GUIDE steps were done through the REST API with curl and then in the browser (Figures 1 to 6). The expected results in the guide all matched.

Test	Result
Register shows active items only; Pumpkin Spice Latte hidden	Pass (not in /api/menu/active, present in the full list as Inactive)
Cashier view of Menu is read-only; Manager view shows the forms	Pass (Figures 2 and 3)
Add category Tea Specials, then item Hojicha Latte at 115.00	Pass, HTTP 201 each
Duplicate category name Coffee	Pass, HTTP 409
Duplicate item name in the same category	Pass, HTTP 409
Blank name, 51-character name, negative price, 3 decimals, 9 integer digits, missing category, unknown category, malformed JSON	Pass, HTTP 400 or 404 with a clear message each time
Deactivate and reactivate an item	Pass, it leaves and returns to the Register list
Order with the same item on two lines	Pass, merged into one line of quantity 2; total 305.50

Change a price afterwards	Pass, the old receipt still shows 115.00 (sold_price is stored per line)
Online order and Daily Sales	Pass. The API gave 1,371.00 and 5 orders, and a direct SQL query on the same day gave the same
Empty order, quantity 0, quantity -3, payment CARD, missing staff, unknown staff, unknown item, inactive item	Pass, rejected each time
Client sends its own price and total in the body	Pass, ignored; the line was stored at 75.50 from the database
DELETE or PUT on an order, DELETE on an item	Pass, HTTP 405 (orders are immutable through the API)
Delete a category that has items / an empty category	Pass, 409 / 204
Text id, unknown id, bad or missing date	Pass, 400 or 404, no stack trace in the response
Direct SQL: negative price, quantity 0, payment CARD, role OWNER, delete a used category, delete a sold item	Pass, each stopped by a constraint

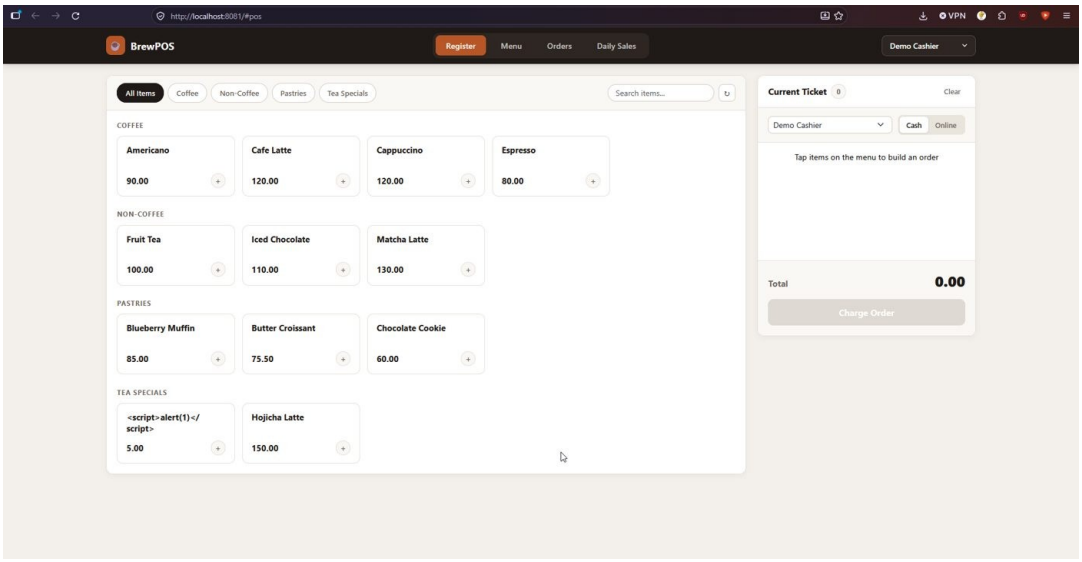


Figure 1. Register. Pumpkin Spice Latte (inactive) is not listed. The extra Tea Specials items came from the tests.

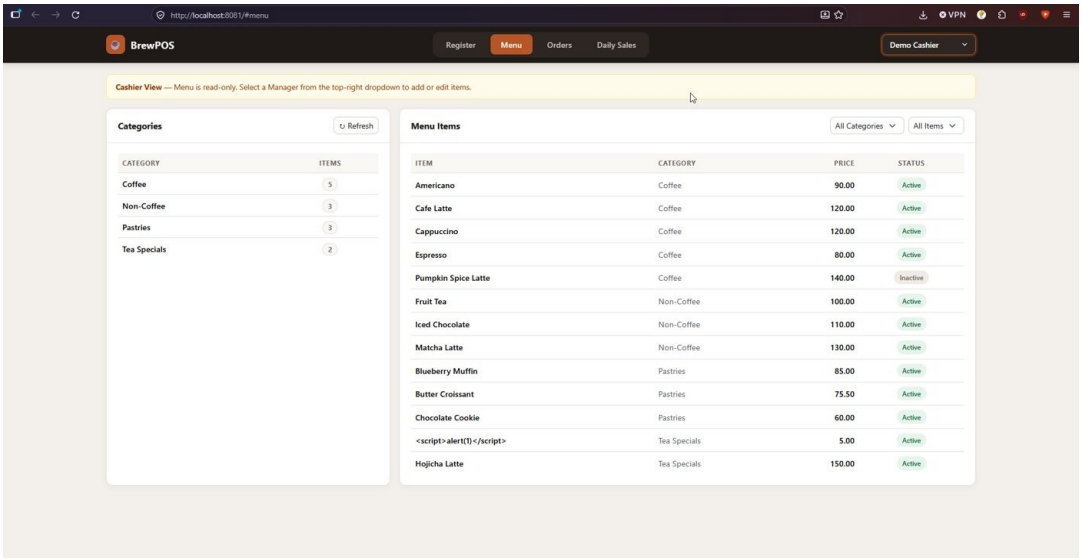


Figure 2. Menu as Demo Cashier: read-only notice, no edit buttons.

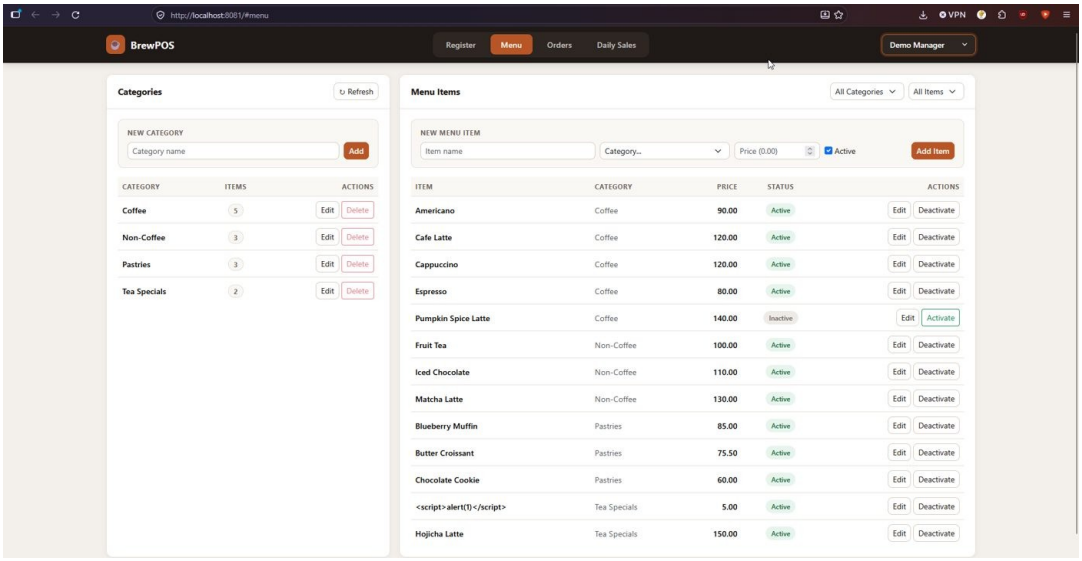


Figure 3. Menu as Demo Manager: New Category and New Menu Item forms and Edit, Delete, Deactivate buttons.

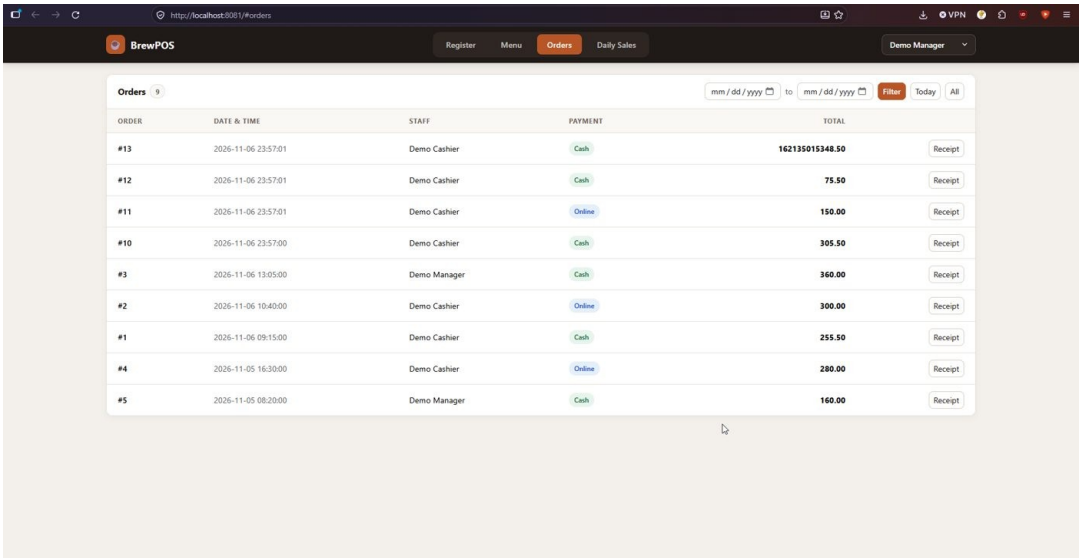


Figure 4. Orders list. Order #13 is the quantity-limit test described in section 6.

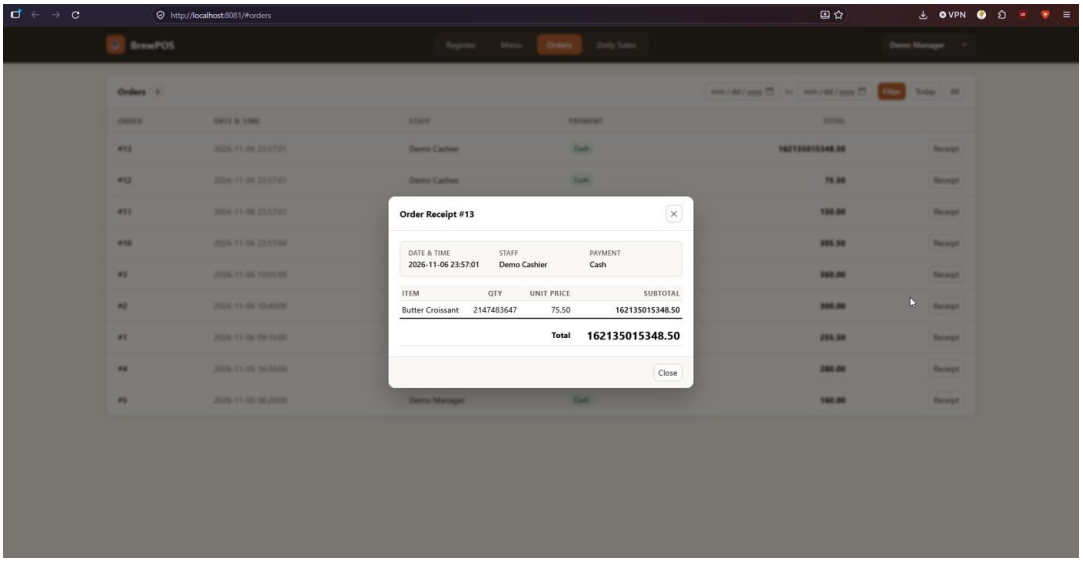


Figure 5. Receipt of order #13: 2,147,483,647 croissants accepted.

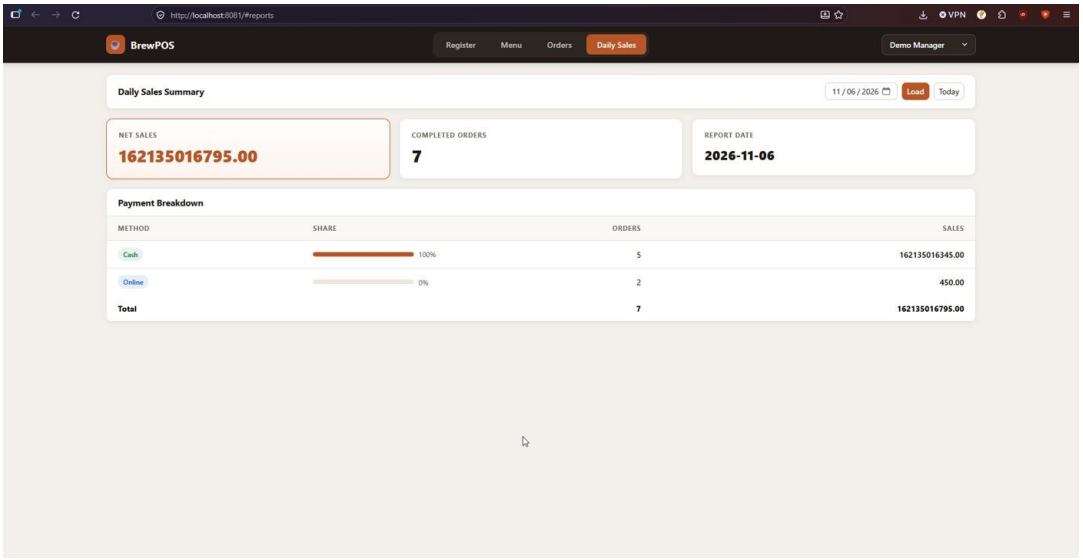


Figure 6. Daily Sales after that order: the net sales total jumps to 162,135,016,795.00.

6. Issues found

#	Issue	Impact and suggestion
1	There is no upper limit on order quantity. An order for 2,147,483,647 of one item was accepted (HTTP 201) and stored, giving a total of 162,135,015,348.50 that also ruined the Daily Sales figure	One mistyped or malicious request corrupts the sales reports for the day. Add a maximum (for example 99) in OrderLineRequest and a CHECK constraint on quantity
2	The API has no authentication. Every call in the tests was made with no cookie, token or password and all of them worked, including creating menu items and recording orders. The Cashier and Manager switch is only a front-end dropdown (a comment in app.js says	Anyone who can reach port 8081 can change the menu and prices. See Activity 4

	authentication is omitted by spec)	
3	Responses carry no security headers (no X-Content-Type-Options, X-Frame-Options or Content-Security-Policy)	Low risk on a classroom system, easy to add with Spring Security headers
4	A menu item named <script>alert(1)</script> is accepted and stored	The front end shows it as plain text (Figures 1 to 3), so it is not exploitable today. Keep escaping on output, and consider rejecting markup in names
5	The .env example and README tell the user to run the app as postgres	Works, but the README itself offers the better brewpos_app account. See Activity 4
6	The automated tests run on the real grp_brewposdb and consume sequence numbers (first real order is #10, categories start at 16)	Harmless, but a separate test database would keep ids tidy

Everything the guide promises works, setup needed no fixes, and the database rules (constraints, immutable history through the API, stored sold price) held up in every test.

7. Feedback for the owners

Issues 1 to 3 are the ones worth fixing. The setup instructions needed no changes.

8. Reproducibility conclusion

Reproducible. From a clean database the README steps gave a working system with no errors, the 19 automated tests passed, and every manual test behaved as documented.

AI usage declaration

- Tool: Claude (Anthropic).
- Tasks: interpreting the results of the testing.
- Validation: the commands were run by the group on its own computer, and every result in this report comes from the saved output files.